Livraisons : stocker, trier, charger

Rapport de TP d'algorithmique — Franklin VERSAYO

1. Choix du langage et découpage du code

J'ai codé le projet en C (norme C11). Ce langage ne fournit ni dictionnaire ni fonction de tri prête à l'emploi : la table de hachage, le tri fusion, le tri par comptage et la programmation dynamique ont donc été écrits de zéro. L'intérêt est qu'aucun coût n'est caché : chaque complexité annoncée dans ce rapport correspond à du code visible. Le C donne aussi un contrôle fin sur la mémoire, que j'ai mis à profit pour ranger la table de la programmation dynamique bit par bit. Seules les entrées-sorties et l'allocation (`malloc` / `free`) viennent de la bibliothèque standard.

Module	Rôle
`src/colis.h`, `src/colis.c`	Définition d'un colis et de la collection qui les regroupe : tableau extensible indexé par une table de hachage (voir 2.1).
`src/tri.h`, `src/tri.c`	Fonctions de tri : fusion pour le poids, la valeur et la destination ; comptage pour la priorité (voir 2.2).
`src/camion.h`, `src/camion.c`	Sélection des colis à charger (sac à dos), avec ou sans prise en compte de la priorité (voir 2.3 et 2.4).
`src/main.c`	Boucle du menu principal, statistiques et banc d'essai des performances.

Deux hypothèses fixent le modèle : un poids est un nombre entier de kilogrammes, au moins égal à 1 (la programmation dynamique indexe un tableau par le poids) ; une priorité est un entier allant de 1, la plus basse, à 5, la plus haute.

2. Décisions algorithmiques

2.1 Retrouver un colis à partir de son numéro

Besoin. Avec jusqu'à 100 000 colis, il faut pouvoir en retrouver un immédiatement par son identifiant, et l'arrivée d'un nouveau colis ne doit pas coûter cher.

Approche retenue. Les colis sont rangés, dans leur ordre d'arrivée, dans un tableau qui double de taille lorsqu'il est plein. À côté, une table de hachage à adressage ouvert fait correspondre chaque identifiant à la position du colis dans ce tableau. La fonction de hachage est multiplicative (méthode de Knuth) : on multiplie l'identifiant par 2654435761 et on garde les bits de poids fort. En cas de collision, on passe à la case voisine (sondage linéaire). Dès que la table est remplie à 50 %, elle est agrandie au double : une recherche visite alors environ 1,5 case en moyenne.

> **Temps :** recherche en O(1) en moyenne, O(n) dans le pire cas théorique où toutes les clés se télescoperaient ; ajout en O(1) amorti, car un agrandissement recopie tout (O(n)) mais ne survient qu'aux puissances de 2 ; affichage complet en O(n).
> **Mémoire :** O(n), soit 48 octets par colis plus une table de 2n à 4n entiers.

Autres pistes. Parcours séquentiel du tableau (O(n) par recherche) ; tableau maintenu trié avec recherche dichotomique (O(log n) pour chercher, mais O(n) pour insérer) ; arbre binaire de recherche équilibré (O(log n) partout, au prix d'un code nettement plus lourd).

Pourquoi celle-ci. C'est l'opération la plus sollicitée, elle doit donc être la plus rapide. Les mesures le confirment : sur 100 000 colis, 1 000 recherches durent 0,005 ms avec la table contre 28 ms par parcours

séquentiel, et la durée avec la table est la même pour 100 colis que pour 100 000.

2.2 Classer les colis selon un critère

Besoin. Afficher les colis classés par poids, valeur, priorité ou destination, dans un sens ou dans l'autre, pour des volumes de 100 à 100 000 colis. L'ordre choisi reste en place pour les affichages suivants.

Approche retenue. Pour le poids, la valeur et la destination (comparée alphabétiquement), j'utilise le tri fusion : le tableau est scindé en deux moitiés, chacune est triée récursivement, puis les deux sont réunies en un seul passage. Le tri porte sur un tableau d'indices et non sur les colis eux-mêmes : chaque déplacement concerne 4 octets au lieu de 48. Ce tri est stable, deux colis égaux gardent leur ordre relatif. Pour la priorité, j'utilise un tri par comptage : un premier passage compte les colis de chaque niveau, des sommes cumulées donnent l'emplacement où commence chaque niveau, et un second passage dépose chaque colis à sa place. Une fois l'ordre obtenu, les colis sont réécrits dans cet ordre et la table de hachage est reconstruite, en O(n).

> **Temps :** fusion en O(n log n) dans tous les cas ($\log_2$ n niveaux de découpage, n opérations par niveau) ; comptage en O(n + k) avec k = 5, c'est-à-dire O(n).
> **Mémoire :** fusion, un tableau auxiliaire en O(n) et une pile de récursion en O(log n) ; comptage, O(k) en plus du tableau résultat.

Autres pistes. Tri rapide : O(n log n) en moyenne, mais $O(n^2)$ dans le pire cas. Tri par insertion : $O(n^2)$, soit environ $5 \cdot 10^9$ opérations pour 100 000 colis, hors de question.

Pourquoi celle-ci. Le tri fusion offre une garantie en O(n log n) indépendante des données : 5 ms pour 100 000 colis. La priorité ne prend que 5 valeurs, ce qui permet au comptage de passer sous la limite Ω(n log n) imposée à tout tri par comparaisons : 0,34 ms pour 100 000 colis.

2.3 Remplir le camion au mieux

Sélectionner des colis dont le poids total ne dépasse pas la capacité C et dont la valeur cumulée est la plus grande possible, c'est exactement le problème du sac à dos 0/1. J'utilise un tableau `meilleur[c]` qui contient la meilleure valeur obtenable avec une capacité c en se limitant aux colis examinés jusque-là. Chaque colis de poids w et de valeur v met à jour ce tableau, c décroissant de C jusqu'à w (le sens décroissant empêche d'utiliser deux fois le même colis) :

```
meilleur[c] ← max( meilleur[c], meilleur[c − w] + v )
```

En parallèle, un bit `pris[i][c]` mémorise si le colis i a fait progresser `meilleur[c]`. Il suffit ensuite de remonter du dernier colis au premier pour savoir lesquels ont été chargés.

Besoin. Déterminer quels colis charger pour transporter le plus de valeur possible sans jamais excéder la capacité.

Approche retenue. Programmation dynamique indexée par la capacité. Le résultat est toujours optimal, puisque la récurrence couvre implicitement tous les sous-ensembles possibles. Pendant le développement, je l'ai confrontée à une énumération exhaustive sur 2 000 jeux de données aléatoires.

> **Temps :** O(n·C). Cette complexité est dite pseudo-polynomiale : elle dépend de la valeur numérique de C, pas seulement du nombre de colis.
> **Mémoire :** O(n·C) bits pour reconstruire la sélection, plus O(C) pour `meilleur[]`. Pour 100 000 colis et C = 50, cela fait 640 Ko, huit fois moins qu'avec un octet par case.

Autres pistes. Algorithme glouton (classer par rapport valeur/poids et prendre tant que ça rentre) : O(n log n), mais sans garantie d'optimalité. Énumération de toutes les combinaisons : optimale mais en $O(2^n)$, déjà 10^{15} cas pour 50 colis. Séparation et évaluation (*branch and bound*) : optimale et souvent rapide, mais exponentielle dans le pire cas.

Pourquoi celle-ci. Exprimée en kilogrammes, la capacité d'un camion reste un petit nombre : O(n·C) s'exécute donc vite (3,2 ms pour 100 000 colis avec C = 50), et c'est la méthode la plus simple qui assure l'optimum exigé par le sujet.

> Vérification sur l'exemple du sujet (A : 8 kg, valeur 20 ; B : 6 kg, 25 ; C : 5 kg, 18 ; D : 4 kg, 14). Avec C = 50, tous les colis tiennent (valeur 77). Avec C = 14, la programmation dynamique choisit A + B, pour une valeur de 45. Un glouton guidé par le rapport valeur/poids aurait pris B (4,17) puis C (3,6), soit 43, en laissant 3 kg vides : il manque l'optimum.

2.4 Intégrer l'urgence des colis

Lorsque l'utilisateur demande de tenir compte de la priorité, la programmation dynamique ne maximise plus la valeur brute mais un score pondéré :

```
score = valeur × (priorité + 3)
```

Besoin. Faire partir en premier les colis urgents, sans pour autant ignorer leur poids ni leur valeur.

Approche retenue. Le score remplace la valeur dans la récurrence. Un colis de priorité 1 compte pour sa valeur × 4, de priorité 3 pour × 6, de priorité 5 pour × 8 : à valeur égale, le plus urgent pèse deux fois plus que le moins urgent. Le poids reste une contrainte stricte. Le score étant entier, les comparaisons ne subissent aucune erreur d'arrondi.

Temps : identique, O(n·C).
Mémoire : identique, O(n·C) bits.

Autres pistes. (a) Comparaison lexicographique : d'abord la somme des priorités, ensuite la valeur. (b) Chargement imposé des colis de priorité 5, puis optimisation sur la place restante. (c) Bonus additif : valeur + β × priorité.

Pourquoi celle-ci. Chacun des trois critères joue un rôle : le poids sert de limite, la valeur et la priorité se combinent dans le score. Un coefficient multiplicatif ne dépend pas de l'ordre de grandeur des valeurs, contrairement au bonus additif (c), et s'interprète facilement. Les deux autres règles ont des effets pervers : (a) peut renoncer à beaucoup de valeur pour un gain de priorité minime, (b) peut saturer le camion de colis urgents lourds et peu rentables.

> Sur l'exemple du sujet, avec les priorités A 5, B 1, C 4, D 3 et une capacité de 13 kg : sans priorité, le camion reçoit B + C (valeur 43) ; avec priorité, il reçoit A + C (valeur 38, mais un score de 160 + 126 = 286, contre 100 + 126 = 226 pour B + C). On cède 5 points de valeur pour que le colis le plus urgent parte.

3. Mesures de performance

Les chiffres ci-dessous proviennent de l'option 7 du menu (statistiques), compilée avec `gcc -O2`, sur des colis générés au hasard (poids de 1 à 30 kg, valeur de 1 à 100). Pour lisser le bruit, chaque opération est relancée jusqu'à totaliser 0,2 s ; le tableau donne la durée moyenne d'une exécution, en millisecondes.

Opération	n = 100	1 000	10 000	100 000
Insertion des n colis	0,004	0,046	0,575	6,733
1 000 recherches, table de hachage	0,003	0,004	0,004	0,005
1 000 recherches, parcours séquentiel	0,019	0,191	2,116	28,125
Tri fusion sur le poids	0,001	0,018	0,472	5,000
Tri par comptage sur la priorité	0,000	0,003	0,031	0,340
Chargement du camion, C = 50 kg	0,003	0,029	0,258	3,226

Recherche par identifiant

- **Coût théorique :** O(1) en moyenne avec la table, O(n) en parcours séquentiel.
- **Ce que montrent les mesures :** avec la table, 1 000 recherches prennent environ 0,005 ms, que la collection compte 100 ou 100 000 colis. Le parcours séquentiel est, lui, multiplié par 10 environ chaque fois que n l'est (de 0,02 à 28 ms).
- **Verdict :** solution adaptée ; pour une recherche sur identifiant exact, on ne peut guère faire mieux.
- **Piste d'amélioration :** des requêtes par plage (par exemple « les colis de 10 à 20 kg ») demanderaient un index ordonné, tableau trié avec dichotomie ou arbre équilibré, en O(log n + k).

Tri

- **Coût théorique :** O(n log n) pour le poids et la valeur, O(n) pour la priorité.
- **Ce que montrent les mesures :** de 10 000 à 100 000 colis, le tri fusion passe de 0,47 à 5 ms, soit ×10,6, cohérent avec le ×12,5 prévu par n log n ; le comptage passe de 0,03 à 0,34 ms, une croissance linéaire.
- **Verdict :** adapté, 5 ms suffisent pour 100 000 colis.
- **Piste d'amélioration :** poids et valeurs sont des entiers bornés ; un tri par base (*radix sort*) les classerait en O(n).

Chargement du camion

- **Coût théorique :** O(n·C) en temps, O(n·C) bits en mémoire.
- **Ce que montrent les mesures :** à capacité fixe, le temps est proportionnel à n (de 0,26 à 3,2 ms entre 10 000 et 100 000 colis, pour C = 50). Il est aussi proportionnel à C : c'est la capacité qui finit par devenir le facteur limitant.
- **Verdict :** adapté tant que la capacité s'exprime en kilogrammes. En grammes ($C = 10^6$) avec 100 000 colis, il faudrait 10^{11} opérations et 12,5 Go de bits, ce qui n'est pas réaliste.
- **Piste d'amélioration :** un camion de capacité C ne peut pas contenir plus de C/w colis de poids w. On peut donc ne conserver, pour chaque poids w, que les C/w colis les plus intéressants : remplacer un colis par un autre de même poids et de meilleure valeur n'abîme jamais la solution. Il reste alors au plus C·(1 + ln C) candidats, environ 190 pour C = 50 au lieu de 100 000, sans perte d'optimalité. Pour des capacités vraiment grandes, on peut arrondir les poids ou passer à un *branch and bound*.

4. Bilan

1. Quel est le choix algorithmique le plus important ? Pourquoi ?

Le recours à la programmation dynamique pour charger le camion. Les autres choix jouent sur la rapidité ; celui-ci est le seul qui décide si la réponse est juste. Sur l'exemple même du sujet, l'approche gloutonne, pourtant naturelle, obtient 43 au lieu de 45, et l'énumération exhaustive devient impraticable passé une trentaine de colis. La programmation dynamique fournit, elle, l'optimum en O(n·C).

2. Quelle partie risque de devenir problématique avec plusieurs millions de colis ?

D'abord le chargement du camion, dont le temps et la mémoire valent n·C : pour 10 millions de colis et C = 1 000, la table de bits atteint 1,25 Go. Le filtrage proposé en partie 3 (au plus C/w colis conservés par poids) suffirait à lever ce blocage. Viennent ensuite la mémoire occupée par la collection elle-même (environ 64 octets par colis, donc 640 Mo pour 10 millions) et les agrandissements du tableau, qui recopient l'intégralité des colis. La recherche, en O(1), et le tri, en O(n log n) (environ 0,6 s pour 10 millions), ne posent pas de difficulté.

3. Avez-vous privilégié la vitesse au détriment de la mémoire, ou inversement ?

J'ai surtout dépensé de la mémoire pour gagner du temps. La table de hachage ajoute 2n à 4n entiers et reste volontairement à moitié vide pour que les recherches soient courtes ; en échange, la recherche passe de O(n) à O(1). De même, garder la table de la programmation dynamique (n·C bits) évite de tout recalculer pour savoir quels colis ont été retenus. Là où c'était possible sans rien perdre, j'ai en revanche économisé : la table est stockée en bits plutôt qu'en octets (facteur 8), et les tris manipulent des indices de 4 octets au lieu de colis de 48 octets.

4. Une autre stratégie aurait-elle pu produire un meilleur résultat pour le camion ?

Pas à objectif égal : la programmation dynamique donne la solution exacte, rien ne peut la dépasser. Une autre méthode pourrait seulement être plus rapide (*branch and bound*, filtrage des candidats) ou presque aussi bonne (glouton). Un résultat « meilleur » supposerait de changer la question elle-même : une autre règle de priorité, la prise en compte du volume, l'ordre de la tournée ou une flotte de plusieurs camions.

5. Avec l'autre langage (Python), changeriez-vous vos choix algorithmiques ?

Les algorithmes resteraient les mêmes, puisque leur complexité ne dépend pas du langage. Seule la mise en œuvre changerait : le `dict` de Python, qui repose sur le même principe, remplacerait ma table de hachage ; les boucles tourneraient 50 à 100 fois moins vite, ce qui rendrait indispensable le filtrage des candidats pour le camion ; enfin, chaque colis occuperait environ 150 à 200 octets sous forme d'objet Python, contre 48 en C. Python aurait en contrepartie l'avantage de supprimer la gestion manuelle de la mémoire (`malloc` / `free`).